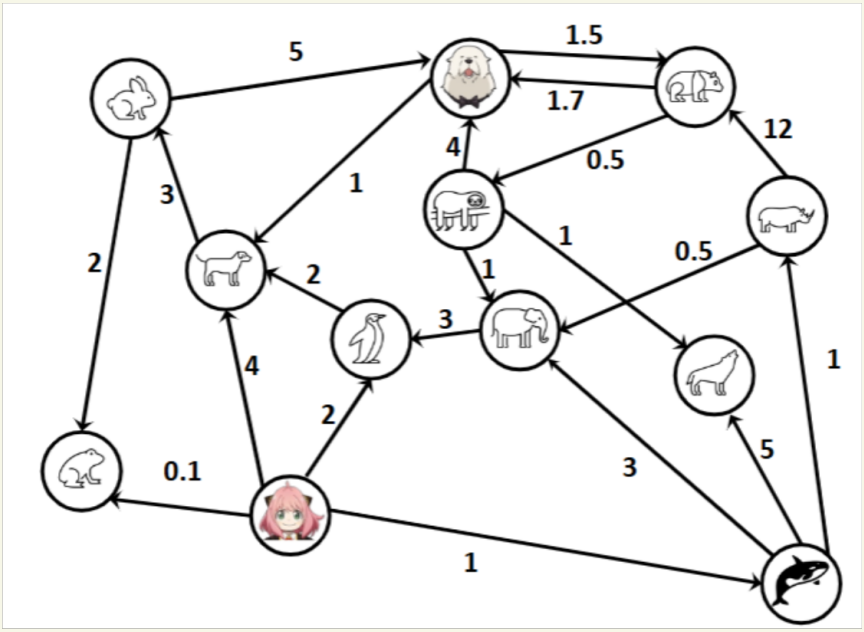
Elijah Konkle

Graphs Assignment

Made with Goodnotes

Graph



1 (text) Dijkstra [15 points] Find the shortest path from Anya (Human) to all other nodes using Dijkstra's (Show your work and intermediate steps)

initialization:

visited: []

priority queue: [(anya, 0)]

step 1:

pop from queue: anya (cost: 0)

mark anya as visited. visited: [anya]

evaluate unvisited neighbors of anya: frog (0 + 0.1 = 0.1), orca (0 + 1 = 1), penguin (0 + 2 = 2), dog (0 + 4 = 4)

push these to priority queue.

priority queue is now: [(frog, 0.1), (orca, 1), (penguin, 2), (dog, 4)]

step 2:

pop from queue: frog (cost: 0.1)

mark frog as visited. visited: [anya, frog]

evaluate unvisited neighbors of frog: none

priority queue is now: [(orca, 1), (penguin, 2), (dog, 4)]

Made with Goodnotes

step 3:

pop from queue: orca (cost: 1)

mark orca as visited. visited: [anya, frog, orca]

evaluate unvisited neighbors of orca: rhino (1 + 1 = 2), elephant (1 + 3 = 4), wolf (1 + 5 = 6)

push these to priority queue.

priority queue is now: [(penguin, 2), (rhino, 2), (dog, 4), (elephant, 4), (wolf, 6)]

step 4:

pop from queue: penguin (cost: 2)

mark penguin as visited. visited: [anya, frog, orca, penguin]

evaluate unvisited neighbors of penguin: dog (2 + 2 = 4, this is not strictly less than the existing cost of 4 so no change)

priority queue is now: [(rhino, 2), (dog, 4), (elephant, 4), (wolf, 6)]

step 5:

pop from queue: rhino (cost: 2)

mark rhino as visited. visited: [anya, frog, orca, penguin, rhino]

evaluate unvisited neighbors of rhino: sloth (2 + 0.5 = 2.5), panda (2 + 12 = 14)

push these to priority queue.

priority queue is now: [(sloth, 2.5), (dog, 4), (elephant, 4), (wolf, 6), (panda, 14)]

step 6:

pop from queue: sloth (cost: 2.5)

mark sloth as visited. visited: [anya, frog, orca, penguin, rhino, sloth]

evaluate unvisited neighbors of sloth: elephant (2.5 + 1 = 3.5, smaller than current 4 so update), wolf (2.5 + 1 = 3.5, smaller than current 6 so update), bond (2.5 + 4 = 6.5)

push and update priority queue.

priority queue is now: [(elephant, 3.5), (wolf, 3.5), (dog, 4), (bond, 6.5), (panda, 14)]

step 7: pop from queue: elephant (cost: 3.5) mark elephant as visited. visited: [anya, frog, orca, penguin, rhino, sloth, elephant] evaluate unvisited neighbors of elephant: none (penguin is already visited) priority queue is now: [(wolf, 3.5), (dog, 4), (bond, 6.5), (panda, 14)]

step 8: pop from queue: wolf (cost: 3.5) mark wolf as visited. visited: [anya, frog, orca, penguin, rhino, sloth, elephant, wolf] evaluate unvisited neighbors of wolf: none priority queue is now: [(dog, 4), (bond, 6.5), (panda, 14)]

step 9: pop from queue: dog (cost: 4) mark dog as visited. visited: [anya, frog, orca, penguin, rhino, sloth, elephant, wolf, dog] evaluate unvisited neighbors of dog: rabbit (4 + 3 = 7) push to priority queue. priority queue is now: [(bond, 6.5), (rabbit, 7), (panda, 14)]

Made with Goodnotes

step 10:

pop from queue: bond (cost: 6.5)

mark bond as visited. visited: [anya, frog, orca, penguin, rhino, sloth, elephant, wolf, dog, bond]

evaluate unvisited neighbors of bond: panda (6.5 + 1.5 = 8, smaller than current 14 so update)

push and update priority queue.

priority queue is now: [(rabbit, 7), (panda, 8)]

step 11:

pop from queue: rabbit (cost: 7)

mark rabbit as visited. visited: [anya, frog, orca, penguin, rhino, sloth, elephant, wolf, dog, bond, rabbit]

evaluate unvisited neighbors of rabbit: none (frog and bond are already visited)

priority queue is now: [(panda, 8)]

step 12:

pop from queue: panda (cost: 8)

mark panda as visited. visited: [anya, frog, orca, penguin, rhino, sloth, elephant, wolf, dog, bond, rabbit, panda]

evaluate unvisited neighbors of panda: none (bond is already visited)

priority queue is now: []

final shortest path costs from anya

frog: 0.1

orca: 1

penguin: 2

rhino: 2

sloth: 2.5

elephant: 3.5

wolf: 3.5

dog: 4

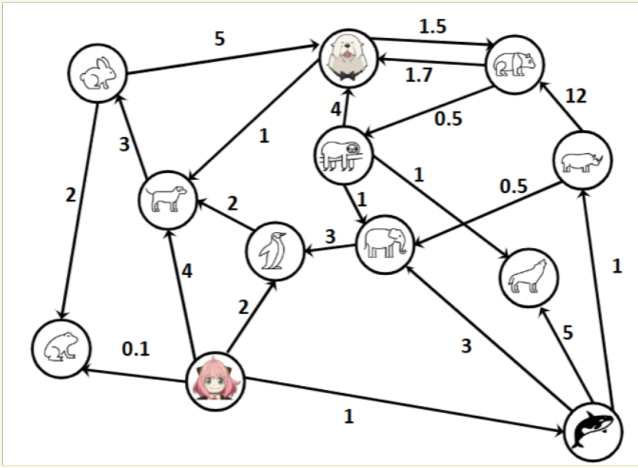
bond: 6.5

rabbit: 7

panda: 8

Made with Goodnotes

2 (text) A* [15 points] Find the shortest path from Anya to all other nodes using A* (Show your work and intermediate steps) **Hint:** Use the geographical location of each node to derive the heuristic to use.



Anya(5,1) | Frog(2,2) | Rabbit(3,10) | Dog(5,6) | Penguin(7,4) | Orca(13,0) | Elephant(10,4) | Wolf(12,4) | Rhino(14,6) | Sloth(9,7) | Bond(9,10) | Panda(12,9)

1)

target: frog (2, 2)

step 1 (start at anya):

frog: g=0.1, h = |2-2| + |2-2| = 0 -> f = 0.1

penguin: g=2, h = |7-2| + |4-2| = 5+2 = 7 -> f = 9

dog: g=4, h = |5-2| + |6-2| = 3+4 = 7 -> f = 11

orca: g=1, h = |13-2| + |0-2| = 11+2 = 13 -> f = 14

next: pop frog (smallest f=0.1). target reached. cost: 0.1

Made with Goodnotes

2)

target: orca (13, 0)
step 1 (start at anya):
orca: g=1, h = |13-13| + |0-0| = 0 -> f = 1
penguin: g=2, h = |7-13| + |4-0| = 6+4 = 10 -> f = 12
frog: g=0.1, h = |2-13| + |2-0| = 11+2 = 13 -> f = 13.1
dog: g=4, h = |5-13| + |6-0| = 8+6 = 14 -> f = 18
next: pop orca (smallest f=1). target reached. cost: 1

3)

target: penguin (7, 4)
step 1 (start at anya):
penguin: g=2, h = |7-7| + |4-4| = 0 -> f = 2
frog: g=0.1, h = |2-7| + |2-4| = 5+2 = 7 -> f = 7.1
dog: g=4, h = |5-7| + |6-4| = 2+2 = 4 -> f = 8
orca: g=1, h = |13-7| + |0-4| = 6+4 = 10 -> f = 11
next: pop penguin (smallest f=2). target reached. cost: 2

4)

target: dog (5, 6)
step 1 (start at anya):
dog: g=4, h = |5-5| + |6-6| = 0 -> f = 4
penguin: g=2, h = |7-5| + |4-6| = 2+2 = 4 -> f = 6
frog: g=0.1, h = |2-5| + |2-6| = 3+4 = 7 -> f = 7.1
orca: g=1, h = |13-5| + |0-6| = 8+6 = 14 -> f = 15
next: pop dog (smallest f=4). target reached. cost: 4

5)

target: rabbit (3, 10)
step 1 (start at anya):
frog: g=0.1, h = |2-3| + |2-10| = 1+8 = 9 -> f = 9.1
dog: g=4, h = |5-3| + |6-10| = 2+4 = 6 -> f = 10
penguin: g=2, h = |7-3| + |4-10| = 4+6 = 10 -> f = 12
orca: g=1, h = |13-3| + |0-10| = 10+10 = 20 -> f = 21
step 2: pop frog (9.1). no edges.
step 3: pop dog (10). evaluate neighbors:
rabbit: g=7, h = |3-3| + |10-10| = 0 -> f = 7
next: pop rabbit (smallest f=7). target reached. cost: 7

Made with Goodnotes

b)

target: rhino (14, 6)

step 1 (start at anya):

orca: g=1, h = |13-14| + |0-6| = 1+6 = 7 -> f = 8

penguin: g=2, h = |7-14| + |4-6| = 7+2 = 9 -> f = 11

dog: g=4, h = |5-14| + |6-6| = 9+0 = 9 -> f = 13

frog: g=0.1, h = |2-14| + |2-6| = 12+4 = 16 -> f = 16.1

step 2: pop orca (8). evaluate neighbors:

rhino: g=2, h = |14-14| + |6-6| = 0 -> f = 2

elephant: g=4, h = |10-14| + |4-6| = 4+2 = 6 -> f = 10

wolf: g=6, h = |12-14| + |4-6| = 2+2 = 4 -> f = 10

next: pop rhino (smallest f=2). target reached. cost: 2

c)

target: sloth (9, 7)

step 1 (start at anya):

penguin: g=2, h = |7-9| + |4-7| = 2+3 = 5 -> f = 7

dog: g=4, h = |5-9| + |6-7| = 4+1 = 5 -> f = 9

orca: g=1, h = |13-9| + |0-7| = 4+7 = 11 -> f = 12

frog: g=0.1, h = |2-9| + |2-7| = 7+5 = 12 -> f = 12.1

step 2: pop penguin (7). evaluate neighbors:

dog: g=4, h = |5-9| + |6-7| = 4+1 = 5 -> f = 9 (no change)

step 3: pop dog (9). evaluate neighbors:

rabbit: g=7, h = |3-9| + |10-7| = 6+3 = 9 -> f = 16

step 4: pop orca (12). evaluate neighbors:

rhino: g=2, h = |14-9| + |6-7| = 5+1 = 6 -> f = 8

elephant: g=4, h = |10-9| + |4-7| = 1+3 = 4 -> f = 8

wolf: g=6, h = |12-9| + |4-7| = 3+3 = 6 -> f = 12

step 5: tie f=8. pop elephant (smallest h=4). evaluates penguin (already visited).

step 6: pop rhino (8). evaluate neighbors:

sloth: g=2.5, h = |9-9| + |7-7| = 0 -> f = 2.5

panda: g=14, h = |12-9| + |9-7| = 3+2 = 5 -> f = 19

next: pop sloth (smallest f=2.5). target reached. cost: 2.5

Made with Goodnotes

8)

target: elephant (10, 4)

step 1 (start at anya):

penguin: g=2, h = |7-10| + |4-4| = 3+0 = 3 -> f = 5

orca: g=1, h = |13-10| + |0-4| = 3+4 = 7 -> f = 8

frog: g=0.1, h = |2-10| + |2-4| = 8+2 = 10 -> f = 10.1

dog: g=4, h = |5-10| + |6-4| = 5+2 = 7 -> f = 11

step 2: pop penguin (5). evaluates dog (no change).

step 3: pop orca (8). evaluate neighbors:

elephant: g=4, h = |10-10| + |4-4| = 0 -> f = 4

rhino: g=2, h = |14-10| + |6-4| = 4+2 = 6 -> f = 8

wolf: g=6, h = |12-10| + |4-4| = 2+0 = 2 -> f = 8

next: pop elephant (smallest f=4). target reached. cost: 4

9)

target: wolf (12, 4)

step 1 (start at anya):

orca: g=1, h = |13-12| + |0-4| = 1+4 = 5 -> f = 6

penguin: g=2, h = |7-12| + |4-4| = 5+0 = 5 -> f = 7

frog: g=0.1, h = |2-12| + |2-4| = 10+2 = 12 -> f = 12.1

dog: g=4, h = |5-12| + |6-4| = 7+2 = 9 -> f = 13

step 2: pop orca (6). evaluate neighbors:

wolf: g=6, h = |12-12| + |4-4| = 0 -> f = 6

rhino: g=2, h = |14-12| + |6-4| = 2+2 = 4 -> f = 6

elephant: g=4, h = |10-12| + |4-4| = 2+0 = 2 -> f = 6

next: tie f=6. pop wolf (smallest h=0). target reached. cost: 6

10)

target: bond (9, 10)

step 1 (start at anya):

penguin: g=2, h = |7-9| + |4-10| = 2+6 = 8 -> f = 10

dog: g=4, h = |5-9| + |6-10| = 4+4 = 8 -> f = 12

orca: g=1, h = |13-9| + |0-10| = 4+10 = 14 -> f = 15

frog: g=0.1, h = |2-9| + |2-10| = 7+8 = 15 -> f = 15.1

step 2: pop penguin (10). evaluates dog (no change).

step 3: pop dog (12). evaluate neighbors:

rabbit: g=7, h = |3-9| + |10-10| = 6+0 = 6 -> f = 13

step 4: pop rabbit (13). evaluate neighbors:

bond: g=12, h = |9-9| + |10-10| = 0 -> f = 12

next: pop bond (smallest f=12). target reached. cost: 12

Made with Goodnotes

1D

target: panda (12, 9)

step 1 (start at anya):

orca: g=1, h = |13-12| + |0-9| = 1+9 = 10 -> f = 11

penguin: g=2, h = |7-12| + |4-9| = 5+5 = 10 -> f = 12

dog: g=4, h = |5-12| + |6-9| = 7+3 = 10 -> f = 14

frog: g=0.1, h = |2-12| + |2-9| = 10+7 = 17 -> f = 17.1

step 2: pop orca (11). evaluate neighbors:

rhino: g=2, h = |14-12| + |6-9| = 2+3 = 5 -> f = 7

elephant: g=4, h = |10-12| + |4-9| = 2+5 = 7 -> f = 11

wolf: g=6, h = |12-12| + |4-9| = 0+5 = 5 -> f = 11

step 3: pop rhino (7). evaluate neighbors:

sloth: g=2.5, h = |9-12| + |7-9| = 3+2 = 5 -> f = 7.5

panda: g=14, h = |12-12| + |9-9| = 0 -> f = 14

step 4: pop sloth (7.5). evaluate neighbors:

bond: g=6.5, h = |9-12| + |10-9| = 3+1 = 4 -> f = 10.5

update elephant: g=3.5, h = |10-12| + |4-9| = 2+5=7 -> f = 10.5

update wolf: g=3.5, h = |12-12| + |4-9| = 0+5=5 -> f = 8.5

step 5: pop wolf (8.5). no outgoing edges.

step 6: tie f=10.5 (bond and elephant). pop bond (smallest h=4).

update panda: g=8, h = 0 -> f = 8

next: pop panda (smallest f=8). target reached. cost: 8

Made with Goodnotes

3 (text) Comparison [5 points] Which algorithm found the shortest path to H in less iterations Dijkstra or A*? (Explain your answer)

I am not quite sure what H is in this problem but I am going to assume that it is the Panda since it is just the farthest from Anya. A* actually found the path in the least amount of iterations Dijkstra's algo took traversing almost the entire graph in order to finally get to the panda. A* only took 7 iterations to get to the panda since the manhattan distance guided it to the rop right and ignored dead end paths.

7 (text) Algorithm Analysis [25 points]
For each of the algorithms you wrote for problems 4-6, explain their time complexity and space complexity using Big-O notation. Explain how you arrived at your answer.

4)

time complexity is O(N^2) where N is the number of nodes because we use a nested loop to build a undirected matrix and since the bfs in the worst case scenario would have to check all N neighbors for each of the N nodes it makes it N^2
space complexity is O(N^2) where N is the number of nodes because we have to create a brand new N by N 2d array each time that we run the program to store the undirected graph which is based on the input size.

5)

time complexity is O(N^4) where N is the number of nodes this is because dfs will follow paths to a depth of 4 and will loop through N neighbors at each of the 4 steps to look for every possible combination.
space complexity is O(N) where N is the number of nodes because we will only create a array of size N to track all the visited nodes and the stack will never go deeper than 4 levels.

Made with Goodnotes

6)

time complexity is O(N^2) where N is the length of the string because we initialize a N by N 2d matrix with zeros and that takes exaclty N^2 operations then to find left and right edges is just sinlge loop which will take O(N) in this case. Lastly we print the matrix row by row and column byu column which is N^2. Since the dominationg factor is N^2 time complexity is N^2

Space complexity is O(N^2) where N is the length of the string and this is because the matrix we build is a N by N 2d array in memory which is eactly N^2.

Made with Goodnotes